

EE 475 Introduction to Image Processing

Homework 07

Boran Aybak Kilic
Student ID: 2022401207

1 Data Type Analysis

The image `book-cover.tif` has a data type of `uint8`. This is a critical detail for processing because `uint8` restricts pixel values to the integer range $[0, 255]$. Any arithmetic operation performed directly on `uint8` data that results in values outside this range will be clipped (saturated). For example, subtracting a higher intensity pixel from a lower one results in 0 rather than a negative value. To perform accurate noise estimation and filtering, it is necessary to convert the images to `double` precision before performing any calculations. This ensures that the full dynamic range of the noise—including negative values relative to the mean—is preserved.

2 Identification of Modifications

To identify the type of noise and modifications applied to `book-cover-gaussian.tif`, I initially calculated the difference between the noisy image and the original.

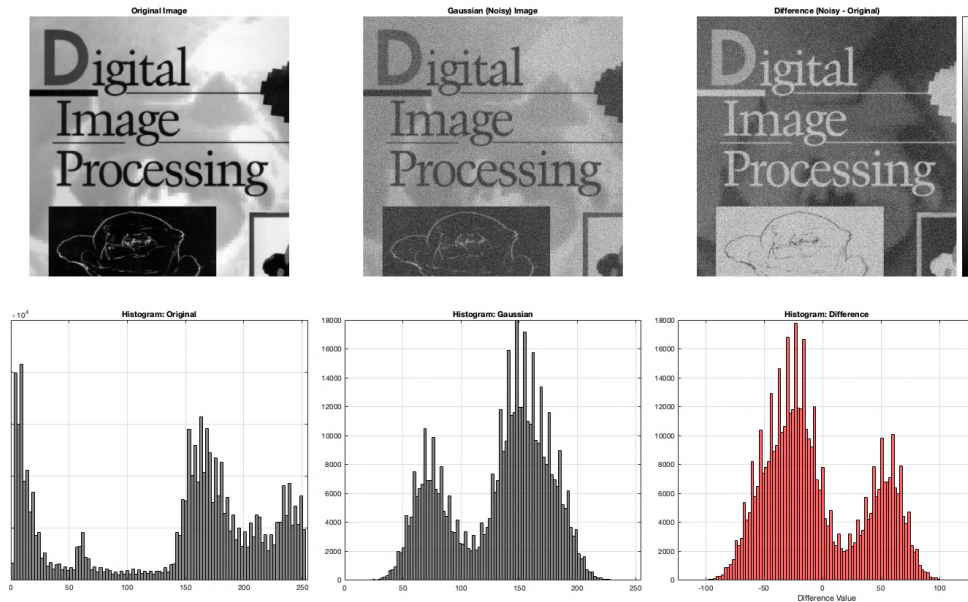


Figure 1: Histograms of the Original, Gaussian (Noisy), and Difference images. Note the bimodal nature of the difference histogram (bottom right).

As shown in Figure 1, the histogram of the difference was bimodal (two peaks) rather than a single Gaussian bell curve centered at zero, which suggests that the noise was not simply additive. Instead, the pixel values were subjected to an affine transformation (linear scaling) before the noise was added. *This was probably done to ensure that pixel values of type `uint8` stayed in the range $[0-255]$ upon the addition of Gaussian noise.*

The relationship can be modeled as:

$$y = \alpha \cdot x + \beta + \eta \quad (1)$$

where y is the noisy pixel value, x is the original pixel value, α is the scaling factor, β is the offset, and η is the Gaussian noise. To confirm this and find the parameters, I plotted the scatter of the original pixel values against the noisy pixel values (Figure 2).

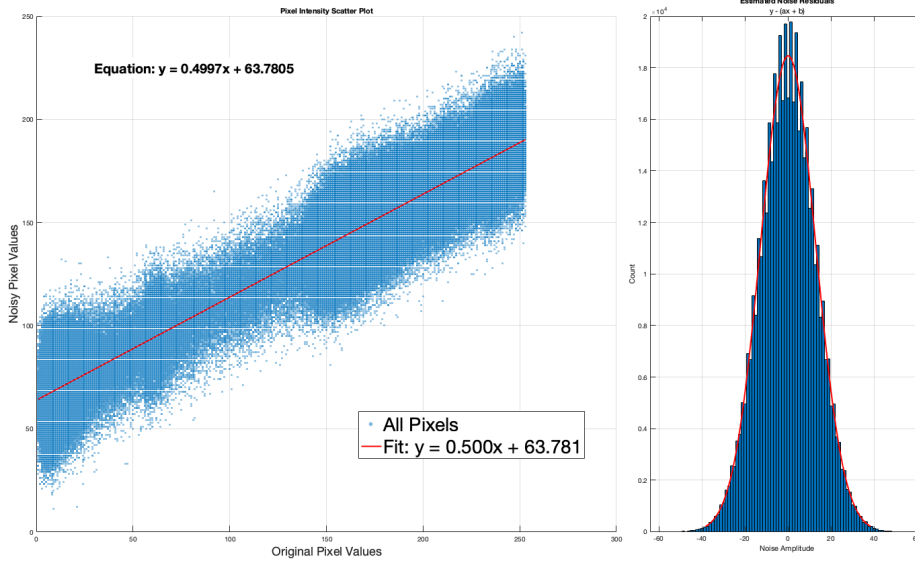


Figure 2: Left: Scatter plot of pixel intensities showing a linear relationship. Right: Histogram of the residuals after removing the linear trend, confirming Gaussian noise distribution.

By performing a linear fit on the pixel data, I determined the modification parameters to be approximately:

$$\alpha \approx 0.5, \quad \beta \approx 63.78$$

Subtracting this linear trend from the noisy image ($y - (\alpha x + \beta) = \eta$) isolated the stochastic component. The histogram of these residuals (Figure 2, right panel) reveals a perfect Gaussian distribution centered at zero. This confirms that the modification consists of a linear intensity scaling followed by additive Gaussian noise.

3 Image Restoration (Denoising)

For filtering, we compared three techniques:

1. Global Gaussian Filter: Applies a 2D convolution with a fixed Gaussian kernel. It performs uniform smoothing across the entire image, effectively reducing noise but blurring high-frequency details like text.

2. Adaptive Local Noise Reduction: Adjusts smoothing based on local statistics using the formula:

$$\hat{f}(x, y) = g(x, y) - \frac{\sigma_\eta^2}{\sigma_L^2(x, y)} [g(x, y) - m_L(x, y)] \quad (2)$$

where σ_η^2 is global noise variance, σ_L^2 is local variance, and m_L is local mean. In flat regions ($\sigma_L \approx \sigma_\eta$), it smooths aggressively. Near edges ($\sigma_L \gg \sigma_\eta$), it preserves the original pixel value.

3. Wiener Filter The Matlab's `wiener2` function estimates the local mean and variance for each pixel neighborhood to minimize the local mean squared error. It assumes additive noise ($H \approx 1$) and adapts the smoothing amount pixel-by-pixel. It's very similar to adaptive filtering except σ_η^2 is replaced by $\sigma_\eta^2 - \sigma_L^2$.

After filtering, the filtered image was rescaled by inverting the linear transformation found in Section 2: $\hat{g} = (g - \beta)/\alpha$.

Performance Metrics: We evaluated performance using Peak Signal-to-Noise Ratio (PSNR) and Normalized Mean Square Error (NMSE).

$$MSE = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} [f(x, y) - \hat{f}(x, y)]^2 \quad (3)$$

$$PSNR = 10 \log_{10} \left(\frac{\max^2(I)}{MSE} \right) \quad (4)$$

where $\max^2(I)$ is the maximum possible pixel value of the image (e.g., 255 for 8-bit images), $f(x, y)$ and $\hat{f}(x, y)$ are the pixel values of original and estimated images, respectively.

$$NMSE = \frac{\sum_{x,y} [f(x, y) - \hat{f}(x, y)]^2}{\sum_{x,y} [f(x, y)]^2} \quad (5)$$

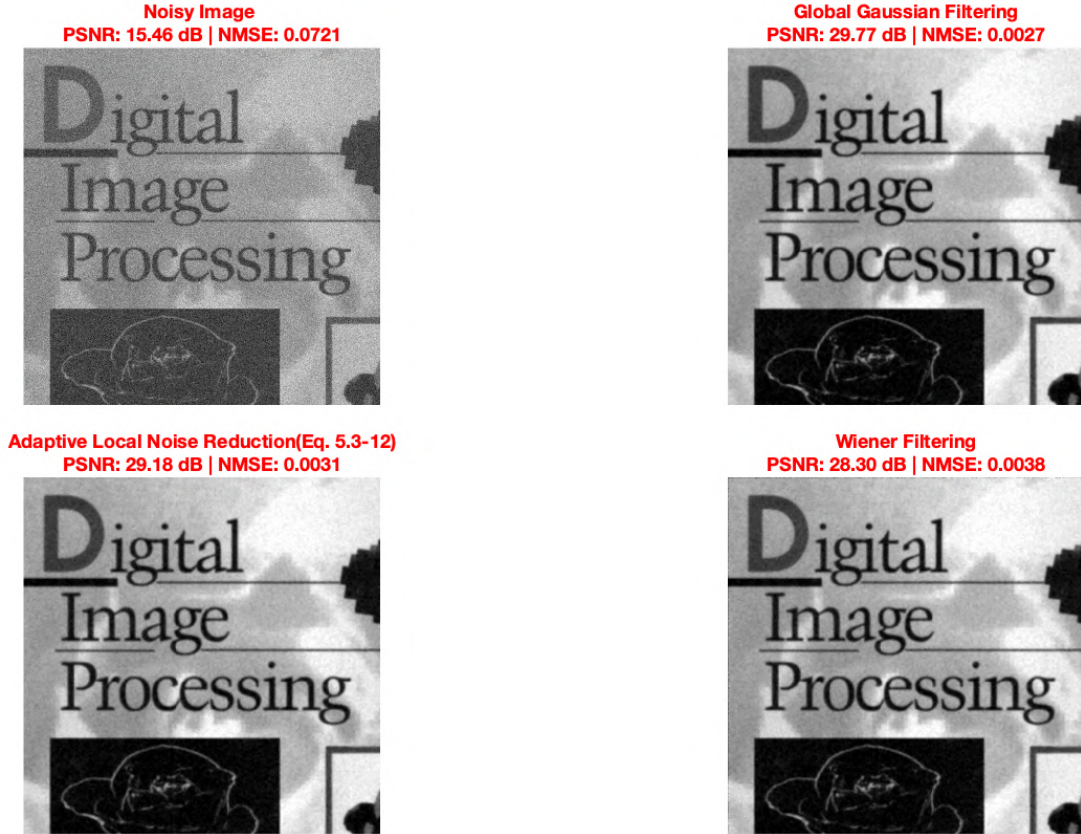


Figure 3: Comparison of denoising results. The Global Gaussian filter yields the highest PSNR but blurs text. Adaptive filters (Wiener) yield sharper text but lower PSNR.

As shown in Figure 3, the Global Gaussian filter achieved the highest PSNR (29.77 dB). This is because it suppressed noise in the large background regions, lowering the global MSE. However, visually, the text appears slightly "blurry" compared to the Wiener and Adaptive filters (approx. 29 dB) with sharper text edges. Their slightly lower PSNR scores result from retaining some noise variance near edges to preserve detail.

4 Motion Deblurring

The blurring observed in `book-cover-blurred.tif` is consistent with linear motion blur, a phenomenon modeled extensively in literature such as *Digital Image Processing* by Gonzalez & Woods. The degradation occurs when the image slips by a distance (Na, Mb) during the exposure time T . The optical transfer function $H(u, v)$ for this uniform linear motion is given by Eq. 5.77 in the textbook:

$$H(u, v) = \frac{T}{\pi(ua + vb)} \sin[\pi(ua + vb)] e^{-j\pi(ua + vb)} \quad (6)$$

where u and v are frequency variables.

To restore the image, we constructed a frequency-domain Wiener filter. Unlike the spatial approximation used in the previous section, this filter uses the explicit degradation function $H(u, v)$ derived above:

$$\hat{F}(u, v) = \left[\frac{H^*(u, v)}{|H(u, v)|^2 + K} \right] G(u, v) \quad (7)$$

Here, K is the noise-to-signal power ratio (NSR) which acts as a regularization parameter to prevent amplification of noise frequencies where $H(u, v)$ is close to zero. Magnitude spectrum of filter is plotted in 4.

Parameter Estimation and Restoration Since the exact motion parameters were unknown, we performed a parameter sweep to maximize the restoration quality (PSNR). We varied the motion components a and b from 0.02 to 0.2 and the regularization parameter K logarithmically. The results of this optimization are visualized in Figure 4.

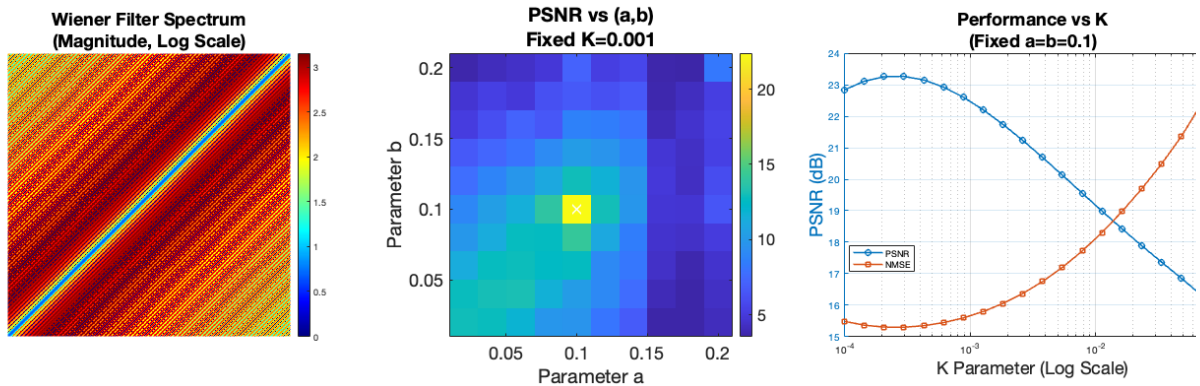


Figure 4: Parameter sweep analysis. Left: Wiener filter spectrum. Center: PSNR heatmap identifying optimal motion parameters $a = b = 0.1$. Right: Sensitivity analysis of metric K , showing a peak performance around $K = 0.05$.

The sweep revealed that the optimal motion parameters are $a = b = 0.1$, corresponding to a diagonal displacement of significant length relative to the image size. The optimal noise-to-signal ratio was found to be approximately $K = 0.0005$. Using these parameters, the final restored image is shown in Figure 5.

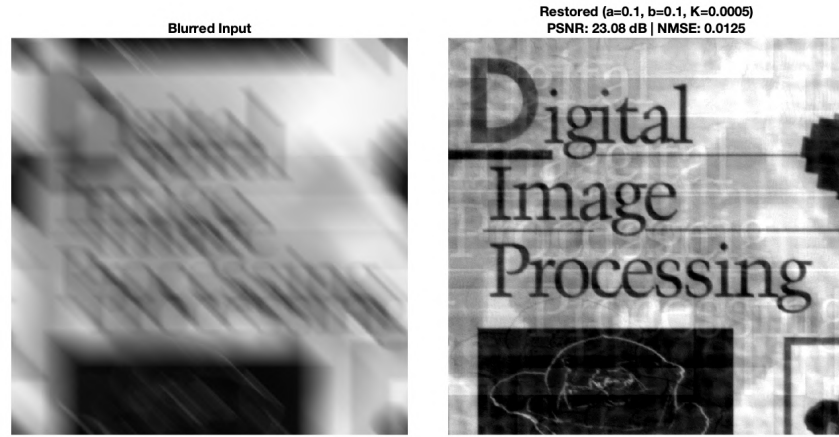


Figure 5: Final restoration using Wiener deconvolution with $a = b = 0.1$ and $K = 0.0005$.

Discussion of Artifacts While the text in the restored image is legible, "ghost" artifacts (faint, displaced copies of the edges) are visible near the boundaries. These occur because the Discrete Fourier Transform (DFT) assumes the image is periodic (repeating infinitely in all directions). The large motion displacement ($a = b = 0.1$) causes pixels from the opposite edge of the periodic extension to "wrap around" into the visible image area during convolution, creating these boundary ringing effects.

A MATLAB Code

```

1 % 1. Load Images
2 img_orig = imread('book-cover.tif');
3 img_gauss = imread('book-cover-gaussian.tif');
4
5 % Convert to double for analysis (Critical for correct subtraction)
6 diff_img = double(img_gauss) - double(img_orig);
7
8 % 3. Linear Fit and Noise Verification
9 x_vals = I_orig(:);
10 y_vals = I_gauss(:);
11
12 % Fit linear model: y = ax + b
13 p = polyfit(x_vals, y_vals, 1);
14 alpha = p(1);
15 beta = p(2);
16 y_fit = polyval(p, x_vals);
17
18 % Calculate Residuals (Estimated Noise)
19 noise_estimated = y_vals - y_fit;
20
21 %% Rest is plotting which is trivial

```

Listing 1: Code for plotting histograms and determining affine transform coefficients

```

1 mg_orig = imread('book-cover.tif');
2 img_gauss = imread('book-cover-gaussian.tif');
3
4 I_orig = im2double(img_orig);
5 I_gauss = im2double(img_gauss);

```

```

6
7 % --- 2. Pre-processing: Invert Linear Scaling ---
8 % We restore the range BEFORE filtering.
9 % Model:  $y = \alpha x + \beta + \text{noise} \rightarrow \hat{x} = (y - \beta) / \alpha$ 
10 alpha = 0.5;
11 beta = 0.25;
12
13 %I_input = (I_gauss - beta) / alpha;
14
15 % Clip to valid range [0, 1] after scaling to avoid artifacts
16 I_input = max(0, min(I_gauss, 1));
17
18 % --- 3. Noise Variance Estimation ---
19 % Since we have the ground truth, we can calculate the exact noise variance
20 % of the input to ensure fair optimal performance for the filters.
21 noise_ref = I_input - I_orig;
22 noise_var_est = var(noise_ref(:));
23
24 % --- 4. Filter Implementations ---
25 % A. Global Gaussian Filter
26 I_res_gauss = imgaussfilt(I_input, 1.5, 'FilterSize', 7);
27 I_res_gauss = (I_res_gauss - beta) / alpha;
28 % B. Wiener Filter (MATLAB Built-in)
29 window = [5 5];
30 I_res_wiener = wiener2(I_input, window, noise_var_est);
31 I_res_wiener = (I_res_wiener - beta) / alpha;
32 % C. Adaptive Local Noise Reduction Filter (From Textbook)
33 % Formula:  $\hat{f} = g - (\text{var\_noise} / \text{var\_local}) * (g - \text{mean\_local})$ 
34 local_mean = imboxfilt(I_input, window); % m_L
35 local_var = stdfilt(I_input, ones(window)).^2; % sigma_L^2
36
37 % Ratio calculation (Limited to 1 to avoid enhancing noise)
38 % If noise_var > local_var, it means the area is purely noise (flat).
39 ratio = noise_var_est ./ (local_var + eps);
40 ratio(ratio > 1) = 1;
41
42 I_res_adaptive = I_input - ratio .* (I_input - local_mean);
43 I_res_adaptive = (I_res_adaptive - beta) / alpha;
44
45 % --- 5. Performance Metrics Functions ---
46 % NMSE = Sum((Ref - Test)^2) / Sum(Ref^2)
47 calc_nmse = @(ref, test) sum((ref(:) - test(:)).^2) / sum(ref(:).^2);
48
49 % Rest of the code is calculating PSNR and NMSE according to definition above
    and plotting
50 %MATLAB's built-in psnr(I_estimate, I_orig) function was used

```

Listing 2: Code for implementing denoising filters on Gaussian

```

1 img_blur = imread('book-cover-blurred.tif');
2 img_orig = imread('book-cover.tif');
3
4 I_blur = im2double(img_blur);
5 [M, N] = size(I_blur);
6
7 % --- 1. Frequency Domain Setup (The Simple Way) ---
8 % Compute FFT and shift DC to the center immediately.
9 % Now (M/2+1, N/2+1) is the center (0,0).
10 G_shifted = fftshift(fft2(I_blur));
11 % Create a natural grid [-M/2 ... M/2]
12 u = -M/2 : ceil(M/2)-1;
13 v = -N/2 : ceil(N/2)-1;
14 [V, U] = meshgrid(v, u);
15

```

```

16 % --- 2. Construct Filter H -using eq. 5.77 from the book--
17 a = 0.1; b = 0.1; T = 1;
18 argument = pi * (U*a + V*b);
19 val_sinc = sin(argument) ./ (argument + eps);
20 % At origin, argument is 0. sin(0)/0 limit is 1.
21 val_sinc(argument == 0) = 1;
22
23 H_shifted = T .* val_sinc .* exp(-1j * argument);
24
25 % --- 3. Wiener Restoration ---
26 K = 0.0005; % Noise-to-Signal ratio chosen by parameter sweep
27
28 % Apply filter directly on the shifted grid
29 W_shifted = conj(H_shifted) ./ (abs(H_shifted).^2 + K);
30 F_hat_shifted = W_shifted .* G_shifted;
31
32 % --- 4. Invert ---
33 % Important: Shift back to "MATLAB format" (DC at corner) before IFFT
34 F_hat = ifftshift(F_hat_shifted);
35
36 % Use real() to discard numerical imaginary noise (approx 1e-15)
37 I_restored = real(ifft2(F_hat));
38
39 %Rest is plotting

```

Listing 3: Code for deblurring image with motion distortion